

Deconvolution

Landman Bester

AIMS South Africa

2026-03-19

Imaging in a Nutshell

Imaging in a Nutshell

Discretized measurement model for imaging

$$y = Rx + \epsilon, \qquad \epsilon \sim \mathcal{N}(0, \Sigma)$$

Distribution of the noise gives

$$\chi^2(x) \propto (y - Rx)^\dagger \Sigma^{-1} (y - Rx) \;\Rightarrow\; \nabla_x \chi^2 = R^\dagger \Sigma^{-1} (y - Rx)$$

Setting the gradient to zero

$$\nabla_x \chi^2 = 0 \;\Rightarrow\; R^\dagger \Sigma^{-1} y = R^\dagger \Sigma^{-1} R\, x \;\Rightarrow\; \boxed{I^D \approx I^{\mathrm{PSF}} \star x}$$

> ⚠ **Not invertible!**
>
> $R^\dagger \Sigma^{-1} R$ is not invertible in general — R is rank-deficient due to incomplete uv-coverage. We need to **deconvolve** I^D.

Method 1 — Tikhonov Regularisation

Tikhonov Regularisation

The simplest fix: add a **penalty** that keeps pixel values small.

$$\underset{x}{\text{minimize}} \quad \underbrace{f(x)}_{\text{data fidelity}} + \lambda \underbrace{\|x\|^2}_{\text{penalty}}$$

Both terms are **smooth** — just differentiate and set the gradient to zero:

$$\nabla_x \left[f(x) + \lambda \|x\|^2 \right] = 0 \implies (H + \lambda I)\, x = I^D$$

A **linear system** — no proximal operator needed, just solve it directly!

Tikhonov — Choosing λ

$$\underset{x}{\text{minimize}} \quad f(x) + \lambda \, \|x\|^2$$

λ controls the trade-off between fitting the data and keeping values small:

λ large	λ small
Over-smoothed, loses detail	Follows noise, amplifies artefacts

Strengths and weaknesses

Fast — one linear solve, no iterations. But $\|x\|^2$ allows negative pixels and penalises all structure equally — it cannot tell signal from noise.

Conjugate Gradient

Problem: H is an $N^2 \times N^2$ matrix — far too large to form or store.

Key insight: we never need to *form* H, only to *apply* it:

$$H\,p \;=\; \text{PSF} \star p \qquad \Longleftrightarrow \qquad \text{two FFTs}$$

Each iteration requires only one matrix-vector product Ap — no matrix storage needed.

Conjugate Gradient — Algorithm

Step	Formula
Init	$x_0 = 0, \quad r_0 = I^D, \quad p_0 = r_0$
Step size	$\alpha_k = r_k^\top r_k \;/\; p_k^\top A\, p_k$
Update x	$x_{k+1} = x_k + \alpha_k p_k$
Update r	$r_{k+1} = r_k - \alpha_k A\, p_k$
Momentum	$\beta_k = \lVert r_{k+1}\rVert^2 / \lVert r_k\rVert^2, \quad p_{k+1} = r_{k+1} + \beta_k p_k$
Repeat	until $\lVert r_k\rVert < \varepsilon$

Converges in at most N^2 steps — in practice **much faster**, because λI improves the conditioning of $H + \lambda I$.

Beyond Smooth Penalties

The Problem with Tikhonov

Tikhonov gives us a clean linear system — but at a cost:

- **Allows negative pixels** — sky brightness cannot be negative, but $\|x\|^2$ has no idea
- **Penalises everything equally** — it shrinks bright sources *and* noise by the same factor; it cannot tell signal from artefact
- **No physical structure** — real sources have sharp edges, point sources, spiral arms — none of which look like the smooth solutions $\|x\|^2$ favours

> **Note**
>
> We need to encode *physical constraints* — like non-negativity — not just a vague preference for small values.

The General Problem

When we cannot (or do not want to) restrict ourselves to smooth penalties:

$$\underset{x}{\text{minimize}} \quad f(x) \; + \; r(x)$$

	$f(x)$	$r(x)$
Statistical	$-\log p(y \mid x)$	$-\log p(x)$
Role	data fidelity	regulariser
Structure	**smooth** — gradient exists	possibly **non-smooth**

When r is non-smooth we **cannot gradient-descend on it** — but we can still minimise it using the *proximal operator*.

The Proximal Operator

$$\text{prox}_{\lambda r}(v) = \underset{x}{\text{argmin}} \quad r(x) + \frac{1}{2\lambda}\|x - v\|_2^2$$

Intuition: find the point x that is simultaneously

- **consistent with the prior** — $r(x)$ is small (or the constraint is satisfied)
- **close to where we already are** — $\|x - v\|^2$ is small

The $\frac{1}{2\lambda}\|x - v\|^2$ tether turns even a hard constraint into a well-posed optimisation.

When r is smooth: $\text{prox}_{\lambda r}(v) \approx v - \lambda \nabla r(v)$ — the proximal operator reduces to a gradient step.

Method 2 — Non-Negative Least Squares

The Indicator Function

We can encode "pixels must be non-negative" as a regulariser:

$$r(x) \;=\; \iota_{+}(x) \;=\; \begin{cases} 0 & \text{if } x_i \geq 0 \;\forall\, i \\ +\infty & \text{otherwise} \end{cases}$$

"$+\infty$" just means **this solution is forbidden** — the optimiser will never choose it.

What is the proximal operator of ι_{+}?

$$\operatorname{prox}_{\iota_{+}}(v) = \operatorname*{argmin}_{x \geq 0} \; \frac{1}{2}\|x - v\|^2 = \max(0,\, v)$$

Just clip negatives to zero — this is the **projection** onto the non-negative orthant, and it is identical to **ReLU**!

Forward-Backward Splitting

To minimise $f(x) + r(x)$ when f is smooth and r may not be, alternate between two steps:

$$\boxed{x_{k+1} = \operatorname{prox}_{\lambda r}\Big(x_k - \gamma \nabla_x f(x_k)\Big)}$$

- **Forward** $\big(x_k - \gamma \nabla_x f\big)$: gradient step on f — moves toward data
- **Backward** $\operatorname{prox}_{\lambda r}(\cdot)$: enforces the constraint / prior

NNLS via Forward-Backward

For **NNLS** ($r = \iota_+$), forward-backward becomes:

$$x_{k+1} = \max\Big(0, \quad x_k - \gamma \nabla_x f(x_k)\Big)$$

Step size

Choose $\gamma < 1/L$ where L is the Lipschitz constant of ∇f.

The Lipschitz Constant

L is the **Lipschitz constant** of ∇f — an upper bound on how fast the gradient changes:

$$\|\nabla f(x) - \nabla f(y)\| \;\le\; L\,\|x - y\| \quad \forall\, x, y$$

For our problem $\nabla f = R^\dagger \Sigma^{-1} R$, so $L = \|R^\dagger \Sigma^{-1} R\|_{\text{op}}$ — the largest eigenvalue.

Power method: repeatedly apply the operator and normalise to find L:

$$v_{k+1} = \frac{A\,v_k}{\|A\,v_k\|}, \qquad L \approx \|A\,v_k\|$$

Converges geometrically — a handful of iterations usually suffice.

Method 3 — Plug-and-Play Denoising

The Prox is a Denoiser

Look at the proximal operator one more time:

$$\text{prox}_{\lambda r}(v) = \underset{x}{\text{argmin}} \underbrace{r(x)}_{\substack{\text{image looks} \\ \text{like the prior}}} + \underbrace{\frac{1}{2\lambda}\|x - v\|^2}_{\substack{\text{stay close} \\ \text{to } v}}$$

v is a noisy intermediate iterate. The prox finds an x that is:

- **plausible** — consistent with whatever prior r encodes
- **data-informed** — close enough to v that we don't discard useful information

This is exactly what a **denoiser** does! So the proximal operator of a prior r **is** a denoiser for that prior — we can swap one for the other.

Plug-and-Play (PnP)

Instead of designing $r(x)$ by hand, **replace** the proximal step with a pre-trained neural denoiser $\mathcal{D}_\sigma$:

$$x_{k+1} = \mathcal{D}_\sigma\Big(x_k - \gamma \nabla_x f(x_k)\Big)$$

The denoiser encodes a prior learned from millions of images — far richer than any formula we could write down.

Sigma annealing — σ controls how aggressively the denoiser cleans up:

- **Early** (σ large): remove large artefacts, establish coarse structure
- **Late** (σ small): refine fine detail without over-smoothing
- **Calibrate** $\sigma_{\min}$ to the actual noise floor — not a natural-image default

Calibrating σ to the Noise Floor

The denoiser expects σ in **pixel units** (Jy/pixel), but we know the noise in **beam units** (Jy/beam). Two steps:

1. Get noise in Jy/beam — project white noise through the adjoint (gridding) operator:

$$n^{\text{beam}} = R^{\dagger}\,\epsilon, \qquad \sigma_{\text{beam}} = \text{std}(n^{\text{beam}})$$

2. Convert Jy/beam → Jy/pixel — run conjugate gradient with a small λ on the noise image:

$$n^{\text{pixel}} = (H + \lambda I)^{-1}\, n^{\text{beam}}, \qquad \sigma_{\text{min}} = \text{std}(n^{\text{pixel}})$$

σ_{min} is the noise floor the denoiser should anneal down to — deconvolution undoes the beam, so the denoiser must match the **deconvolved** noise

Equivariant Denoising

The sky has no preferred orientation — but DRUNet was trained on natural images and treats rotations unequally.

> **D4 group averaging:** run the denoiser on all 8 rotations/reflections of the image, then average the results back.

This eliminates orientation bias at the cost of 8× the forward passes — a worthwhile trade for astronomical imaging.

> ⓘ **Note**
>
> `EquivariantDenoiser(DRUNet(...))` in deepinv does this in one line.

The deepinv Library

deepinv

A PyTorch library for imaging inverse problems — data fidelity terms, pretrained denoisers, solvers, and metrics out of the box.

deepinv.github.io/deepinv

```python
from deepinv.optim.linear import conjugate_gradient
from deepinv.models import DRUNet, EquivariantDenoiser

# Tikhonov via conjugate gradient
x_tik = conjugate_gradient(
    A=lambda x: psf_conv(x) + lam * x,
    b=dirty_image)

# Plug-and-Play forward-backward
denoiser = EquivariantDenoiser(
    DRUNet(pretrained="download"))
for i in range(n_iter):
    x = denoiser(x - step * grad_f(x), sigma[i])
```

Thanks!

lbester@sarao.ac.za